

Edge Assisted Multi-Camera Vehicle Tracking Framework for Real-Time and Scalable Deployment

Yuqiang Lin^a, Sam Lockyer^a, Shucheng Zhang^c, Florian Stanek^b, Markus Zarbock^b, Adrian Evans^a, Wenbin Li^a,
Yinhai Wang^c, Nic Zhang^{a,*}

^aUniversity of Bath, United Kingdom

^bStarwit Technologies GmbH, Germany

^cUniversity of Washington, United States

Abstract

Cameras are a core sensing modality in modern intelligent transportation systems (ITS), providing rich visual information of road-user activities. Multi-Camera Vehicle Tracking (MCVT) leverages this data to reconstruct vehicle trajectories across a camera network, enabling applications such as traffic flow prediction and optimisation. However, existing MCVT studies mostly emphasize on the accuracy but overlook real-time performance and scalability. These two aspects are critical for the real-world deployment of these applications and have become especially challenging in city-scale applications, as the number of cameras in a city traffic network grows. To address this gap, we propose an **Edge-Assisted, Scalable and Efficient-MCVT (EASE-MCVT)** framework explicitly designed for real-time throughput and scalable deployment. More specifically, EASE-MCVT follows an edge-server architecture. On the edge side, each camera stream is processed in real time through object detection, single-camera tracking, geo-mapping, and feature extraction, while only lightweight metadata, including vehicle locations and appearance features, are transmitted to the central server for cross-camera association. To improve both tracking performance and system efficiency, EASE-MCVT is further optimized from both the algorithmic and system perspectives. On the algorithmic side, we introduce a dynamic workload scheme for tracklet-level feature extraction on edge devices, a server-side single-camera re-match module to reconnect fragmented tracklets, and a self-supervised camera link model that learns spatial-temporal constraints to accelerate and stabilize cross-camera association. On the system side, we integrate production-oriented data engineering components to standardize deployment and data exchange for large-scale operation. To the best of our knowledge, EASE-MCVT is the first MCVT framework that is designed to address both real-time performance and scalability in a distributed edge-server setting. We have evaluated the framework on the RoundaboutHD and CityFlow datasets. Results have successfully demonstrated its real-time throughput while maintaining competitive tracking accuracy, paving the way for future city-wide real-time traffic management.

Keywords: Multi-Camera Vehicle Tracking, Edge Computing, Computer Vision, Smart Cities, Traffic monitoring

1. Introduction

The rapid development of urbanization has posed new challenges for transportation authorities in maintaining efficient, safe, and sustainable traffic operations. As road networks become more complex and the number of vehicles continues to grow, conventional traffic management methods are often insufficient to handle growing transportation demands (Wang et al., 2019). To address these challenges, leveraging information from existing traffic cameras has become a practical and cost-effective approach. These cameras continuously capture rich visual information about road environments that can be processed through advanced computer vision technologies and provide data-driven insights for intelligent traffic management.

Among these techniques, multi-camera vehicle tracking (MCVT) has emerged as a particularly promising approach. The main goal of MCVT is to associate vehicle trajectories across different cameras, ensuring consistent

*Corresponding author.

vehicle identities between different fields of view (FOV) (Amosa et al., 2023). This capability enables the reconstruction of complete vehicle trajectories across the monitored area. When applied to multiple vehicles, MCVT can effectively generate a comprehensive representation of the traffic flow. With the reconstructed traffic flow data available, many downstream applications can be achieved, including traffic flow prediction, suspect vehicle tracking, traffic anomaly detection and so on. Collectively, these applications can be used as components to build a safer, smarter, and more efficient transportation system.

Generally, the MCVT problem can be divided into two stages (Holla et al., 2025): (i) *the intra-camera process*, which detects and tracks vehicles within the same camera view across consecutive frames; and (ii) *the inter-camera process*, which associates vehicle identities across different camera FOVs. A major challenge in MCVT arises from the small inter-class variability and large intra-class variability among vehicle identities. The same vehicle can appear dramatically different under varying lighting conditions, viewpoints, or occlusions, which complicates cross-camera matching. Conversely, vehicles of the same model often exhibit only subtle appearance differences, making re-identification difficult even for humans (Tang et al., 2019). Luckily, in recent years, many researchers have developed different methods to address those challenges e.g. (Liu et al., 2021; Qian et al., 2020; Yang et al., 2022; Lin et al., 2025c,a), achieving promising accuracy and establishing a solid theoretical foundation for the MCVT task. However, most of existing approaches rely on complex deep neural network or multi-stage algorithmic pipelines—such as performing computationally heavy offline re-matching after single-camera tracking, some researches even combining outputs from multiple tracking algorithms for accuracy (Li et al., 2022). Such approaches take high computational cost and would be limited to offline operations, usually long after a traffic event has occurred. Moreover, when the system needs to scale up to a city-wide deployment with large numbers of cameras, the computational burden massively increases for the intra-camera process, it increases linearly but the inter-camera identity association will increase quadratically. Those above challenges limits in both the real-time performance and the scalability of existing MCVT systems. Overall, there is a substantial gap between academic research and real-world implementation of MCVT framework.

With the rapid advancement of mobile hardware capabilities and the growing adoption of edge computing technologies, we can see some feasible real-time and scalable solutions to solve the multi-camera vehicle tracking (MCVT) problem (Gong et al., 2023; Ke et al., 2020). Generally, edge computing enables the distribution of computational tasks from a central server to devices located closer to the data source. In the settings, using this edge-based architecture allows the entire intra-camera process to be deployed and executed at the computational unit near the camera (Ke et al., 2022). Specifically, each edge node independently conducts the object detection and single camera tracking tasks; and then only the processed metadata—such as vehicle IDs, appearance features, bounding box location and timestamps—are transmitted to the central server for the subsequent inter-camera association stage (Zhou et al., 2021). This distributed design offers several advantages: (i) it reduces the central server’s computational burden by distributing most processing across parallel edge nodes, thereby improving scalability; (ii) it lowers communication bandwidth requirements, as transmitting compact metadata is far more efficient than streaming raw video; and (iii) it improves data privacy and storage efficiency by limiting the transmission and storage of sensitive visual content. Together, these benefits provide a strong foundation for real-time, scalable, and privacy-preserving MCVT systems; however, translating this architectural potential into a practical MCVT system still requires task-specific design from both the algorithmic and system perspectives. To the best of our knowledge, existing studies have not systematically addressed how to build a real-time and scalable MCVT framework under an edge-server architecture with joint consideration of both aspects. Thus, we considering multiple perspectives requirements when we design the EASE-MCVT framework:

(i) Real-time requirements. Achieving end-to-end real-time performance requires coordinated optimization on both the edge and server sides:

- *Edge-side requirements.* To reduce the computational burden on the central server, major intra-camera workloads, particularly object detection and feature extraction, should be shifted to the edge side. At the same time, these modules must remain efficient enough for deployment on resource-constrained edge hardware. Therefore, task-specific optimization of these two modules is essential in our design.
- *Server-side requirements.* On the server side, one major challenge is that metadata arriving from the edge nodes may vary in quality or be received asynchronously due to factors such as network latency and frame loss. Therefore, the server-side association process must be designed to operate reliably under asynchronous and imperfect input conditions.

(ii) **Systems-level scalable deployment requirements.** Enabling real-world city-scale deployment also requires system-level optimization through data-engineering design and deployment standardization:

- *Data requirements.* A practical deployment must support structured data exchange between distributed components with low latency, high throughput, and robustness over the network. It also requires stable data interfaces and explicit schemas to ensure reliable communication and reduce integration friction.
- *Deployment and maintenance requirements.* City-scale systems require repeatable installation, configuration, updates, and monitoring across heterogeneous hardware and distributed sites. They must also support long-term maintainability so that the system can evolve without disrupting existing components.

(iii) **Accuracy requirements.** A reliable city-scale MCVT framework must maintain strong tracking performance under diverse real-world conditions, including viewpoint variation, occlusion, illumination changes, and heterogeneous camera layouts. Therefore, in addition to real-time and deployment considerations, the framework must incorporate task-specific designs that improve the robustness of both intra-camera processing and cross-camera association.

Motivated by these requirements, we propose EASE-MCVT, an edge-server MCVT framework designed for real-time operation and city-scale deployment. fig. 1 illustrates the overall workflow of EASE-MCVT. The main contributions of this work are as follows:

- *Edge-server architecture for deployment-oriented MCVT.* We propose an edge-server architecture for multi-camera vehicle tracking, in which the intra-camera pipeline is executed on edge nodes while the inter-camera association process is handled by a central server. This design enables parallel processing of camera streams, reduces the communication burden through metadata-only transmission, and provides a practical foundation for scalable deployment.
- *Algorithmic optimization for real-time and robust operation.* To support real-time throughput and reliable cross-camera tracking, we optimize several core modules in the proposed framework. Specifically, we optimize the object detection stage for edge deployment, introduce a dynamic workload scheme for tracklet-level feature extraction, design an online triggering scheme for asynchronous cross-camera association, and incorporate a single-camera re-match module together with a self-supervised camera link model to improve association robustness and accuracy.
- *System-level implementation for practical deployment.* To improve deployability, communication efficiency, and long-term maintainability, we integrate production-oriented system components into the framework, including Docker for modular deployment, Redis for low-latency message passing, Protocol Buffers for standardized data exchange, and TimescaleDB for persistent trajectory storage and management.
- *Comprehensive framework evaluation.* We evaluate EASE-MCVT on the *RoundaboutHD* and *CityFlow* datasets. The results show that EASE-MCVT achieves an IDF1 score of 71.63% on *RoundaboutHD*, which is the best among the compared methods, and 55.17% on *CityFlow*, while maintaining real-time performance under the evaluated settings. In addition to the overall MCVT results, we provide detailed analyses of the core modules in terms of both accuracy and runtime stability, together with ablation studies that quantify the contribution of the proposed add-on components.

The rest of the paper is structured as follows: Section 2 provides a comprehensive literature review for the multi-camera tracking problem and researcher’s effort to develop a real-time multi-camera tracking pipeline. Section 3 outlines the algorithm details for EASE-MCVT framework with evaluation on different dataset listed on Section 4. Finally, Section 5 summarises the main findings and proposing direction for future research.

2. Background

As already noted, the MCVT problem can be divided into two stages: (i) intra-camera process and (ii) inter-camera process. The intra-camera process aims to detect vehicles and maintain consistent identities across frames from a single camera (Amosa et al., 2023). (1) *Vehicle Detection.* Current deep-learning based methods can be mainly

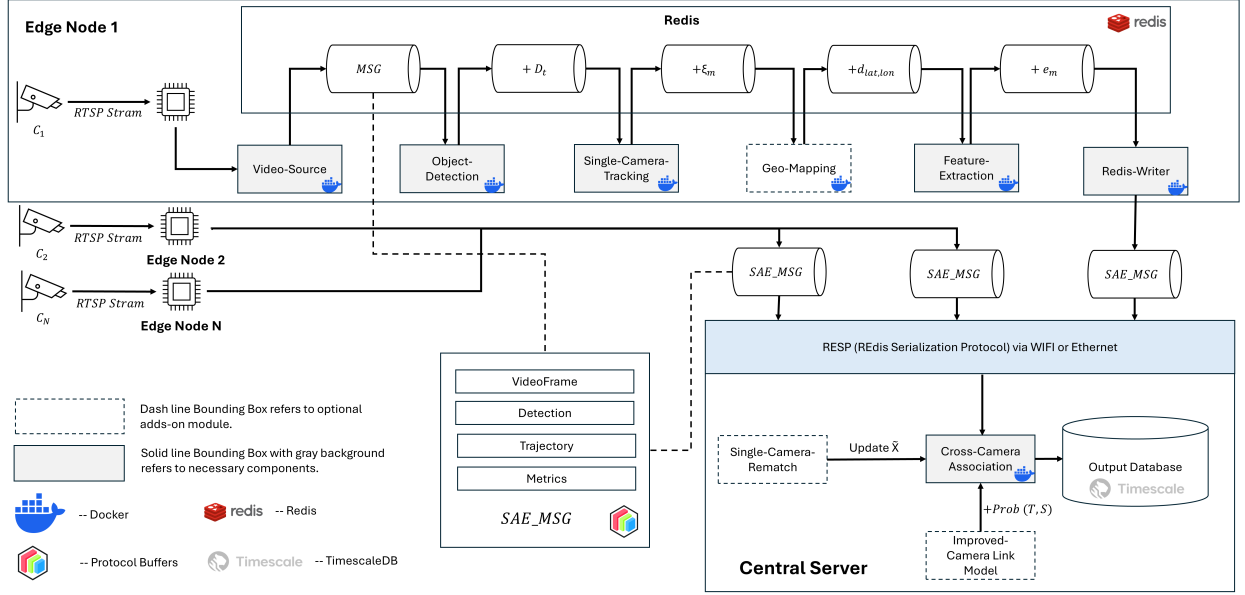


Figure 1: Overview of the proposed EASE-MCVT edge-server framework. (I) A set of N edge nodes, each connected to one or more cameras, consumes live RTSP streams and performs the intra-camera pipeline (detection, single-camera tracking, geo-mapping, and feature extraction). Only lightweight metadata (tracklets, spatial coordinates, and appearance embeddings) are serialized and transmitted via RESP to (II) the central server, which performs inter-camera association and maintains global multi-camera trajectories.

classified into two categories: one-stage detectors, such as YOLO family (Tian et al., 2025; Varghese and Sambath, 2024), and two-stage detectors, such as Fast R-CNN Girshick (2015). Since 2020, one-stage detectors—particularly the YOLO family have become dominant in both academic research and real-world applications due to the balance of their speed and accuracy. Very recently, some researchers also tried to adopt the powerful pre-trained multi-modal large language model (MLLM) for the zero-shot object detection task (Zang et al., 2025; Bai et al., 2023). Due to the huge size and corresponding computational cost of MLLM, YOLO family is still the first choice for edge-based real-world application. (2) *Single Camera Tracking*. After acquiring the object location, the single camera tracking module aims to identify the temporal consistency of like objects. Existing SCT methods can be broadly categorized into two categories: deep-embedding based and non-embedding based. For the deep embeddings based algorithm e.g DeepSORT (Wojke et al., 2017) and BotSort (Aharon et al., 2022), the similarity of objects across different frames depends both on the bounding-box (Bbox) Intersection over Union (IoU) and deep embeddings similarity. The calculation of deep embeddings improves the robustness of the tracker to handle occlusion and frame loss, but introduces more computational cost in the SCT process. For the non-embedding based methods, such as SORT (Bewley et al., 2016) and Bytetrack (Zhang et al., 2022). Those methods only rely on positional prediction and matching, which is lightweight and more suitable for resource-constraint edge-based real-world application. (3) *Feature Extraction*. In an edge-server MCVT setting, appearance features are typically computed on the edge and sent to the server to support cross-camera association. Once detections are produced and temporally linked by SCT, the next step is to compute compact and discriminative appearance features that can be sent to the downstream cross-camera association. One of the biggest challenges for feature extraction is the need for huge computational resources for robust deep embeddings. Many existing approaches (Yang et al., 2022; Li et al., 2022) rely on complex CNN backbones such as ResNet (He et al., 2016) to extract discriminative features. Some offline pipelines further increase complexity by combining multiple backbones (e.g., Transformers and several ResNet variants) to obtain stronger embeddings (Zhang et al., 2024), which is impractical for real-time deployment on embedded hardware.

The inter-camera process refers to linking vehicle identities across different cameras. Depending on how an identity is represented, this linking can be formulated at two levels. In image-based vehicle ReID, each identity is represented by one or more still images captured by different cameras, and the goal is to match these image instances across views. In video-based ReID, which is MCVT, each identity is represented by a sequence of observations pro-

duced by the intra-camera pipeline (e.g., tracklets or trajectories), and the goal is to associate these trajectories across cameras to form continuous cross-camera vehicle tracks. Image-based ReID typically ignores temporal continuity, which simplifies the matching problem but decouples performance from intra-camera tracking quality. Consequently, the image-based ReID focuses on extracting representative deep embeddings and different matching methods, some well-known researches includes PROVID (Liu et al., 2017), Git (Shen et al., 2023) and Going beyond real data Zheng et al. (2020). Those studies provide a solid foundation for the inter-camera MCVT process. With computer vision technologies becoming more mature, researchers’ insights start focusing more on the video-based vehicle ReID problem. Compared with image-based ReID, video-based settings require processing many more observations per identity which provides more visual information, but also increases the complexity of the problem. To mitigate these challenges, some researchers (Yang et al., 2022) introduce a heavy SCT module, e.g. (Li et al., 2022) combines the results from three different SCT modules which brings huge computational load. And (Yang et al., 2022)’s SCT module takes very heavy deep embeddings as input, combining multiple SCT algorithms together and introducing an offline re-link module, this makes the SCT module over-engineered. Other studies leverage manually defined zones to encode spatio-temporal relationships within and across adjacent cameras (Liu et al., 2021; Wu et al., 2021). While such zone-based constraints prove to be effective in controlled or small-scale environments, they are difficult to scale: zone definitions require manual effort, and any change in camera placement or orientation may necessitate re-labeling. Overall, existing work highlights a central challenge for practical deployment—achieving a reliable balance between tracking accuracy, computational efficiency, and system scalability.

Although no existing system fully addresses the real-world Multi-Camera Vehicle Tracking (MCVT) problem, several studies have made progress toward enabling real-time and deployable vision-based ITS. For instance, Zhang and Izquierdo (2019) proposes a real-time multi-camera tracking framework that processes video data frame by frame instead of waiting for each batch to complete, achieving real-time performance on the DukeMTMC dataset (Ristani et al., 2016). More recently, Zhuang et al. (2024) developed a real-time MCVT framework that takes continuous video streams as input and demonstrates real-time performance on both the CityFlow dataset (Tang et al., 2019) and their custom dataset. However, the real-time capability of these systems heavily depends on high performance hardware, and performance quickly becomes a bottleneck as the number of cameras increases. To mitigate this limitation, several studies (Liu et al., 2023; Ke et al., 2020) have explored the edge-server architecture that allows the distribution of computational workloads for practical ITS deployment. Although these systems are not designed specifically for MCVT, their real-world deployments provide evidence that edge-based architectures can improve scalability for vision-driven ITS applications. Other closely related research is cooperative edge-based vehicle ReID. For instance, (Yang et al., 2023) developed an edge-enabled vehicle ReID system deployed in real traffic conditions; however, it operates at the clip level rather than maintaining full trajectories. The work most closely related to EASE-MCVT is Gaikwad and Karmakar (2021), which applies edge computing to multi-camera pedestrian tracking to achieve scalable and real-time performance. However, in real traffic networks, pedestrians exhibit much lower speeds, smaller activity ranges, and smaller scales compared to vehicles, making vehicle tracking a far more challenging problem.

3. EASE-MCVT Framework

3.1. EASE-MCVT Overview

Problem Definition: Consider a camera network with N cameras indexed by $C = \{1, \dots, N\}$. Each camera $i \in C$ provides a video stream (or a set of clips) over a time interval $[t_1, t_2]$. Let $\mathcal{T}$ denote the discrete timestamp set within $[t_1, t_2]$ (e.g., frame times). At each time $t \in \mathcal{T}$, the detector produces a set of vehicle detections $\mathcal{D}_{i,t} = \{d_{i,t}^{(k)}\}_{k=1}^{n_{i,t}}$, where each detection is represented by metadata $d_{i,t}^{(k)} = (t, \mathbf{b}_{i,t}^{(k)}, s_{i,t}^{(k)})$. Here $\mathbf{b}_{i,t}^{(k)} \in \mathbb{R}^4$ is the bounding box (e.g., (x, y, w, h)) and $s_{i,t}^{(k)} \in [0, 1]$ is the detection confidence. After we get the detection, the single camera tracker will group detections into a set of local tracklets (trajectories) for each camera, denoted by $\Xi_i = \{\xi_{i,m}\}_{m=1}^{M_i}$ for $i \in C$. Each local tracklet $\xi_{i,m}$ is an ordered sequence of detections $\xi_{i,m} = \{d_{i,t}^{(k)} \mid t \in \mathcal{T}_{i,m} \subseteq \mathcal{T}\}$ and is associated with a *local* identity label $\ell_{i,m}$ that is only meaningful within camera i . Notably, in our framework, each local tracklet is further represented by a trajectory-level appearance embedding $\mathbf{e}_{i,m} \in \mathbb{R}^d$, yielding the augmented tracklet representation $\tilde{\xi}_{i,m} = (\xi_{i,m}, \mathbf{e}_{i,m})$. Collecting all augmented tracklets across the camera network, we define $\tilde{\Xi}_i = \{\tilde{\xi}_{i,m}\}_{m=1}^{M_i}$ and $\tilde{\mathcal{X}} = \bigcup_{i \in C} \tilde{\Xi}_i$ as the set of all augmented local tracklets observed during $[t_1, t_2]$. The goal of MCVT is then to associate these augmented

tracklets across cameras and recover *global* vehicle identities by estimating a global identity assignment $\phi : \tilde{\mathcal{X}} \rightarrow \mathcal{G}$, where $\mathcal{G}$ is the set of global vehicle identities, such that for any two augmented tracklets $\tilde{\xi}_{i,m}, \tilde{\xi}_{j,n} \in \tilde{\mathcal{X}}$, $\phi(\tilde{\xi}_{i,m}) = \phi(\tilde{\xi}_{j,n})$, if and only if $\tilde{\xi}_{i,m}$ and $\tilde{\xi}_{j,n}$ correspond to the same physical vehicle. The resulting global trajectory set can be written as $\mathcal{T}^{\text{global}} = \{\mathcal{T}_g\}_{g \in \mathcal{G}}$ with $\mathcal{T}_g = \{\tilde{\xi} \in \tilde{\mathcal{X}} \mid \phi(\tilde{\xi}) = g\}$, i.e., each global trajectory $\mathcal{T}_g$ is a cross-camera collection of local tracklets which will share the same global identity.

Framework Architecture. We design EASE-MCVT to solve the MCVT problem defined above while meeting the practical requirements of real-time throughput and city-scale scalability. From experience, most of the computational load happens in the intra-camera process; as the number of cameras increases, executing all intra-camera processing on a single central machine quickly becomes infeasible. This motivates an edge-server architecture in which each camera stream is processed locally on an edge device (NVIDIA Jetson AGX Orin in our deployment), and only lightweight metadata is transmitted to a central server for inter-camera association. An overview of the architecture is shown in fig. 1. On the edge side, each camera connects directly to its corresponding edge node, which consumes the RTSP stream and converts it into a structured format via the *video-source* module. The stream is then processed by *object detection* and *single camera tracking* to generate local tracklets, followed by *geo-mapping* and *feature extraction* to attach geo-coordinates and trajectory-level appearance embeddings to each tracklet. The resulting messages are serialized and published by the *Redis-Writer* to the central server over network. On the server side, we perform *cross-camera association* to link tracklets from different cameras and assign global identities. To further improve robustness and accuracy, we also include two optional modules: *single-camera rematch* and an *improved camera-link model*. Finally, the global identities $\mathcal{G}$ and associated trajectories are stored in a database. The details of each module are described in the rest of this section.

3.2. Edge Node Algorithm Design

3.2.1. Video-Source Raw Video Pre-Process

To facilitate downstream intra-camera processing, we design a *video-source* module to pre-process the incoming RTSP stream. For each frame, the module (i) captures the image, (ii) attaches basic metadata including the camera/source ID, timestamp, and frame resolution, and (iii) encodes the frame payload as JPEG-compressed bytes for efficient transport. The resulting frame message is then structured using a protobuf and serialized for transmission to subsequent modules in the edge pipeline.

3.2.2. Real-time Object Detection

3.2.2.1. YOLO detector. In recent years, the YOLO family of detectors has become dominant in both academic research and real-world applications due to their speed and accuracy. For our deployment, we evaluated three recent YOLO variants pre-trained on the COCO dataset (Lin et al., 2014): YOLOv8 (Varghese and Sambath, 2024), YOLOv11 (Khanam and Hussain, 2024), and YOLOv12 (Tian et al., 2025), which differ in architecture and computational cost. Through our inspection, all three models exhibited frequent missed detections for vehicles viewed from an overhead perspective, as well as for small and distant vehicles. To address this domain gap, we fine-tuned these YOLO models on the RoundaboutHD object-detection dataset (Lin et al., 2025b). After comparing detection accuracy and inference latency, we selected the fine-tuned YOLOv11 model and converted it to TensorRT (NVIDIA, 2026) to enable more efficient inference on the edge device.

3.2.2.2. Motion Detector. We observe that traffic scenes often include extended intervals with little or no vehicle activity—particularly during late-night hours—making it inefficient to run the full detection and tracking pipeline continuously on edge hardware. To reduce unnecessary computation and power consumption, we introduce a lightweight motion detector placed upstream of the YOLO module to filter those near empty scenes. For efficiency, we adopt the classical MOG2 background-subtraction method (Zivkovic, 2004) and estimate the number of foreground (moving) pixels per frame. The activation threshold τ_{fg} is set as a fixed fraction of the image area, which normalizes the criterion across different resolutions. If the foreground pixel count exceeds τ_{fg} , we activate the YOLO-based detector; otherwise, the edge pipeline skips downstream processing for that frame.

3.2.3. Single Camera Tracking

In the SCT module, detections from each incoming frame are associated with existing in-memory tracklets to maintain temporal identity consistency. Using the open-source, pluggable tracking library BOXMOT (Broström,

2025), we evaluated several state-of-the-art trackers, including both embedding-based and non-embedding-based methods, in terms of FPS and IDF1. Based on these results and the trade-off between runtime efficiency and robustness on edge hardware, we select ByteTrack (Zhang et al., 2022) as the default SCT algorithm in EASE-MCVT. ByteTrack is embedding-free and therefore avoids per-object feature extraction overhead, which is beneficial on resource-constrained edge devices. We also keep the interface modular so that alternative trackers can be swapped in when different operating conditions or accuracy-latency requirements apply.

3.2.4. Geo-Mapping

The geo-mapping module converts the pixel-space location of each detection into a real-world position, expressed either in GPS coordinates (latitude, longitude, elevation) or in a camera space coordinate system. This spatial information provides additional evidence for multi-camera association, especially when cameras have overlapping or partially overlapping fields of view (FOV). The mapping is defined by a camera transformation model parameterized by camera specifications (e.g., field of view, heading, tilt, roll, and mounting height) and the camera’s geo-location. In practice, these parameters can be obtained from sensor specifications and site measurements, or calibrated using map priors (e.g., from satellite imagery) when the camera location is known. We implement this transformation using the camera model provided by Gerum et al. (2017):

$$\mathbf{p}_{i,t}^w = \text{Cam}(x_{i,t}^{\text{pix}}, y_{i,t}^{\text{pix}}, h_{i,t}; \boldsymbol{\theta}_i), \quad (1)$$

where $\mathbf{p}_{i,t}^w = [\text{lat}_{i,t}, \text{lon}_{i,t}, z_{i,t}]^\top$ denotes the estimated world position of the vehicle at time t in camera i , $(x_{i,t}^{\text{pix}}, y_{i,t}^{\text{pix}})$ is the detection center in pixel coordinates, $h_{i,t}$ is a height prior used to resolve depth, $\boldsymbol{\theta}_i$ denotes the camera parameters, and $\text{Cam}(\cdot)$ is the camera transformation model.

3.2.5. Feature Extraction

3.2.5.1. Tracklet-Level Feature Representation. Running deep-neural-network based feature extraction on every frame can be computationally demanding on resource-constrained edge devices. To reduce this overhead while maintaining robust appearance feature for downstream association, EASE-MCVT performs feature extraction at the tracklet level rather than the frame level. As a result, the edge node buffers the per-frame tracklet updates and triggers feature extraction after a tracklet is completed within the camera’s field of view.

We adopt a lightweight ResNeXt-50 backbone (Xie et al., 2017) and fine-tuned it on RoundaboutHD ReID dataset to keep the balance between accuracy and inference speed on edge hardware. For a completed local tracklet $\xi_{i,m}$, the feature extractor outputs a matrix of frame-level embeddings $\{\mathbf{e}_{i,m}^{(k)}\}_{k=1}^{N_{i,m}}$, where $\mathbf{e}_{i,m}^{(k)} \in \mathbb{R}^{2048}$ and $N_{i,m}$ is the number of frames selected for feature extraction from this tracklet. We aggregate these embeddings using confidence-weighted averaging followed by ℓ_2 normalization to obtain a tracklet-level embedding:

$$\hat{\mathbf{e}}_{i,m} = \frac{\sum_{k=1}^{N_{i,m}} c_{i,m}^{(k)} \mathbf{e}_{i,m}^{(k)}}{\left\| \sum_{k=1}^{N_{i,m}} c_{i,m}^{(k)} \mathbf{e}_{i,m}^{(k)} \right\|_2}, \quad (2)$$

where $\hat{\mathbf{e}}_{i,m} \in \mathbb{R}^{2048}$ is the normalized tracklet embedding and $c_{i,m}^{(k)}$ is the detection confidence of the k -th cropped instance used as the aggregation weight. After aggregation, the edge node transmits lightweight metadata—tracklet detection, the frame level embeddings $\{\mathbf{e}_{i,m}^{(k)}\}_{k=1}^{N_{i,m}}$, and $\hat{\mathbf{e}}_{i,m}$ —to the central server for inter-camera association, without sending raw images.

3.2.5.2. Dynamic Workload Scheme. Although tracklet-level extraction reduces computation compared to frame-by-frame extraction, the workload still varies with traffic density and tracklet length. To improve its stability on variable traffic conditions, we control the number of frames processed per tracklet by introducing an adaptive subsampling factor $K_{i,m}$. For camera i , we define the total number of cropped instances to be processed within time window Δ as

$$V_i(\Delta) = \sum_{m=1}^{n_i(\Delta)} \left\lceil \frac{|\mathcal{T}_{i,m}|}{K_{i,m}} \right\rceil, \quad (3)$$

where $n_i(\Delta)$ is the number of tracklets completed on camera i during Δ , $\mathcal{T}_{i,m}$ denotes the set of frames in tracklet $\xi_{i,m}$, and $|\mathcal{T}_{i,m}|$ is its length in frames. Intuitively, a larger $K_{i,m}$ results in stronger temporal subsampling and therefore a smaller processing workload. In implementation, our goal is to keep the system to be real-timed while using the smallest possible $K_{i,m}$ to preserve feature quality. Thus, we count the number of cropped instances for each tracklet, and further define the $K_{i,m}$ as:

$$K_{i,m} = \max\left(1, \left\lceil \frac{|\mathcal{T}_{i,m}|}{Q_{\max}(\Delta)} \right\rceil\right), \quad (4)$$

where $Q_{\max}(\Delta)$ is an estimated hyperparameter representing the maximum number of cropped instances that can be processed for a tracklet within the allowed time window Δ under the target hardware setting without causing queue overflow in our test conditions. Under this policy, longer tracklets are subsampled more aggressively, while shorter tracklets retain denser visual observations. As a result, the per-tracklet feature-extraction workload is bounded according to the available processing capacity, which helps maintain stable queue behavior in steady operation.

3.3. Central Server Design

3.3.1. Single-Camera Re-Match

SCT in the intra-camera stage can be degraded by illumination changes, occlusions, irregular vehicle motion, and occasional frame loss. These effects often lead to track identity switches, especially when lightweight, non-embedding-based trackers are used on edge devices. To reduce the impact of imperfect SCT on downstream cross-camera association without increasing edge-side computation, we introduce a *single-camera re-match* module on the central server, where greater compute resources are available. This module refines the SCT output by re-linking fragmented tracklets that are likely to belong to the same vehicle within the same camera. For the re-match process, we apply a rule-based merging policy. Given two candidate tracklets Tracklet_A and Tracklet_B from the same camera, where Tracklet_A temporally precedes Tracklet_B , we merge them if and only if the following three conditions are all satisfied:

$$\text{Merge}(\text{Tracklet}_A, \text{Tracklet}_B) = \mathbb{I}[\Delta t < T_{\text{th}} \wedge \Delta c < D_{\text{th}} \wedge \Delta e < F_{\text{th}}], \quad (5)$$

where $\mathbb{I}[\cdot]$ is an indicator function, and

$$\Delta t = t_B^{\text{start}} - t_A^{\text{end}}, \quad \Delta c = \|\mathbf{c}_B^{\text{start}} - \mathbf{c}_A^{\text{end}}\|_2, \quad \Delta e = 1 - \max(\cos_sim(\bar{\mathbf{e}}_A, \bar{\mathbf{e}}_B), \cos_sim(\mathbf{e}_A^{\text{end}}, \mathbf{e}_B^{\text{start}})). \quad (6)$$

Here t_A^{end} and t_B^{start} denote the end time of Tracklet_A and the start time of Tracklet_B , respectively, and T_{th} is the maximum allowed temporal gap. The vectors $\mathbf{c}_A^{\text{end}}, \mathbf{c}_B^{\text{start}} \in \mathbb{R}^2$ are the bounding-box centers at the end of Tracklet_A and the start of Tracklet_B , and D_{th} is the spatial distance threshold. The vectors $\bar{\mathbf{e}}_A, \bar{\mathbf{e}}_B \in \mathbb{R}^d$ are the ℓ_2 -normalized averages of the per-detection embeddings within each tracklet, while $\mathbf{e}_A^{\text{end}}$ and $\mathbf{e}_B^{\text{start}}$ are the embeddings of the last detection of Tracklet_A and the first detection of Tracklet_B . Finally, $\cos_sim(\cdot, \cdot)$ computes cosine similarity and F_{th} is the appearance-distance threshold.

We adopt a conservative merging strategy by using tight thresholds ($T_{\text{th}}, D_{\text{th}}, F_{\text{th}}$), prioritizing precision over recall. This choice is because we believe an incorrect re-merge irreversibly corrupts the identity history and can mislead downstream cross-camera association, whereas leaving tracklets unmerged is less harmful. The updated tracklets set $\tilde{\mathcal{X}}$ are then forwarded to the *cross-camera association* module.

3.3.2. Self-Supervised Camera Link Model

To improve cross-camera association accuracy, many prior studies (e.g., (Hsu et al., 2019)) introduce the spatial-temporal constraints derived from road topology and travel-time statistics to filter infeasible matches and narrow down the searching space when linking tracklets across adjacent cameras. While effective, manually defining such constraints is labor-intensive and becomes impractical at city scale. Following on from our previous work (Lin et al., 2025c), we adopt a *self-supervised camera link model (CLM)* that learns camera-to-camera spatial-temporal relationships offline without requiring human annotations. The model consists of three major steps.

(1) Identifying entry and exit zones within each camera. We first estimate a set of entry and exit zones for each camera view. Intuitively, entry zones correspond to regions where vehicles frequently appear (i.e., where tracklets start), while exit zones correspond to regions where vehicles frequently disappear (i.e., where tracklets end). For camera i , let $\mathcal{Z}_i$ denote the set of all discovered zones, with entry zones $\mathcal{Z}_i^{\text{entry}} \subseteq \mathcal{Z}_i$ and exit zones $\mathcal{Z}_i^{\text{exit}} \subseteq \mathcal{Z}_i$. Zones

are obtained in two stages: (i) we cluster the start pixel locations and end pixel locations of all tracklets in camera i ; (ii) for each cluster, we define the zone region by fitting an axis-aligned bounding box that encloses the cluster points. Each fitted region is treated as a candidate zone. Finally, we label each zone as an entry or exit zone according to the entry/exit density of tracklet starts and ends within that region.

(2) Pairing entry–exit zones across adjacent cameras. After obtaining entry and exit zones for each camera, the next step is to establish correspondences between zones in adjacent camera pairs. For an adjacent camera pair (i, j) , let $\mathcal{Z}_i^{\text{exit}} = \{\mathbf{z}_{i,m}^{\text{exit}}\}_{m=1}^{N_i^{\text{exit}}}$ and $\mathcal{Z}_j^{\text{entry}} = \{\mathbf{z}_{j,n}^{\text{entry}}\}_{n=1}^{N_j^{\text{entry}}}$. We compute a zone-pairing score for each candidate pair using the $\text{ZonePair}(\cdot)$ function in (Lin et al., 2025c):

$$s_{i,j}(\mathbf{z}_{i,m}^{\text{exit}}, \mathbf{z}_{j,n}^{\text{entry}}) = \text{ZonePair}(\mathbf{z}_{i,m}^{\text{exit}}, \mathbf{z}_{j,n}^{\text{entry}}), \quad \forall m, n. \quad (7)$$

We then select the best-matching correspondence by

$$(\hat{m}, \hat{n}) = \arg \max_{m,n} s_{i,j}(\mathbf{z}_{i,m}^{\text{exit}}, \mathbf{z}_{j,n}^{\text{entry}}). \quad (8)$$

We apply the same procedure in the reverse direction (i.e., pairing $\mathcal{Z}_j^{\text{exit}}$ with $\mathcal{Z}_i^{\text{entry}}$). As a result, each adjacent camera pair yields directional entry–exit correspondences that serve as spatial constraints for downstream association: when matching tracklets from camera i to camera j , we only consider candidates that end in the selected exit zone of camera i and begin in the selected entry zone of camera j .

(3) Camera-link transition-time estimation. For each directional paired zone correspondence $(i \rightarrow j)$ (i.e., an exit zone in camera i paired with an entry zone in camera j), we estimate the distribution of vehicle transition times using Gaussian kernel density estimation (KDE).

Transition-time sample collection. Following Lin et al. (2025c), we first construct a set of high-confidence cross-camera matches offline to obtain reliable transition-time samples. Specifically, we select candidate tracklets that (i) end in the paired exit zone of camera i and (ii) begin in the paired entry zone of camera j . We then perform a preliminary cross-camera association using strict appearance thresholds to favor precision (i.e., minimizing false matches). The resulting matched tracklet pairs are treated as pseudo ground truth, and for each matched pair we compute the transition time as

$$\tau_{i,j}^{(\ell)} = t_{j,n}^{\text{start}} - t_{i,m}^{\text{end}}, \quad (9)$$

where $t_{i,m}^{\text{end}}$ is the end time of the matched tracklet in camera i and $t_{j,n}^{\text{start}}$ is the start time of the matched tracklet in camera j . Collecting all samples yields $\{\tau_{i,j}^{(\ell)}\}_{\ell=1}^{L_{i,j}}$.

KDE estimation. Given the observed samples, we estimate a continuous transition-time density:

$$\hat{p}_{i,j}(t) = \frac{1}{L_{i,j}h} \sum_{\ell=1}^{L_{i,j}} \mathcal{K}\left(\frac{t - \tau_{i,j}^{(\ell)}}{h}\right), \quad (10)$$

where $h > 0$ is the bandwidth and $\mathcal{K}(\cdot)$ is the Gaussian kernel

$$\mathcal{K}(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right). \quad (11)$$

The learned density $\hat{p}_{i,j}(t)$ provides a temporal constraint for online association. In summary, the self-supervised camera link model allows us to restrict candidate cross-camera association to those that (i) satisfy the learned zone correspondence and (ii) have transition times that are sufficiently likely under $\hat{p}_{i,j}(t)$.

3.3.3. Cross-Camera Association

Cross-camera association links tracklets from different cameras that correspond to the same physical vehicle. We perform association over the road-topology graph: for each topologically adjacent camera pair (i, j) , we construct a pairwise cost matrix and solve a one-to-one assignment. Applying the same procedure to all adjacent pairs yields global identities across the network.

Cost matrix construction. For an adjacent camera pair (i, j) , let $\tilde{\Xi}_i = \{\tilde{\xi}_{i,m}\}_{m=1}^{M_i}$ and $\tilde{\Xi}_j = \{\tilde{\xi}_{j,n}\}_{n=1}^{M_j}$ denote the augmented tracklet sets from cameras i and j , respectively, where each $\tilde{\xi}_{i,m} = (\xi_{i,m}, \mathbf{e}_{i,m})$ consists of a local tracklet and its trajectory-level embedding. For a candidate pair $(\tilde{\xi}_{i,m}, \tilde{\xi}_{j,n})$, we define the travel-time gap as

$$\Delta t_{m,n} = t^{\text{start}}(\xi_{j,n}) - t^{\text{end}}(\xi_{i,m}), \quad (12)$$

and compute the pairwise association score as

$$\text{Cost}(\tilde{\xi}_{i,m}, \tilde{\xi}_{j,n}) = \delta \cos_sim(\mathbf{e}_{i,m}, \mathbf{e}_{j,n}) + \epsilon \log \hat{p}_{i,j}(\Delta t_{m,n}), \quad (13)$$

where $\cos_sim(\cdot, \cdot)$ denotes cosine similarity between embeddings, $\hat{p}_{i,j}(\cdot)$ is the transition-time density learned by the camera-link model for the directional link $(i \rightarrow j)$, and δ and ϵ weight the appearance and temporal terms, respectively. Candidate pairs that violate the learned zone-based spatial constraint will be filtered before cost construction when it is available.

The full cost matrix between cameras i and j is

$$\mathbf{C}_{i,j} = \begin{bmatrix} \text{Cost}(\tilde{\xi}_{i,1}, \tilde{\xi}_{j,1}) & \cdots & \text{Cost}(\tilde{\xi}_{i,1}, \tilde{\xi}_{j,M_j}) \\ \vdots & \ddots & \vdots \\ \text{Cost}(\tilde{\xi}_{i,M_i}, \tilde{\xi}_{j,1}) & \cdots & \text{Cost}(\tilde{\xi}_{i,M_i}, \tilde{\xi}_{j,M_j}) \end{bmatrix}. \quad (14)$$

Cost matrix solving and online triggering. For each adjacent camera pair (i, j) , we solve a one-to-one assignment on $\mathbf{C}_{i,j}$ to associate tracklets that likely correspond to the same physical vehicle. We adopt greedy matching as default due to its low latency: candidate pairs are ranked by $\text{Cost}(\cdot, \cdot)$ in descending order, and matches are accepted sequentially while enforcing the one-to-one constraint. We also provide the Hungarian algorithm as an alternative when an optimal assignment is preferred. To avoid spurious associations, we reject matched pairs whose cost is higher than a predefined threshold. After solving all adjacent camera pairs, we consolidate the resulting matches to update the global identity assignment $\phi : \tilde{\mathcal{X}} \rightarrow \mathcal{G}$ and the corresponding global identity set $\mathcal{G}$.

In principle, cross-camera association could be triggered whenever the tracklet set $\tilde{\mathcal{X}}$ is updated. However, solving assignments for every incremental update introduces unnecessary overhead and is not required in real-world deployments. Instead, we run association periodically every α frames to balance computational efficiency and end-to-end latency. The results of each association are written to the database, where they are merged with historical records to extend existing trajectories and register newly formed trajectories.

3.4. Data Engineering Implementation

To support real-world deployment rather than simulation-only evaluation, we implement EASE-MCVT through adopting our industrial partner’s SAE-ENGINE pipeline, which provides production-oriented frameworks including *Docker*, *Redis*, *Protocol Buffers (Protobuf)*, and *TimescaleDB*. These components standardize deployment, data transmission, and persistent storage across heterogeneous edge and server hardware.

Specifically, we use *Docker* to containerize each module as a docker image, enabling reproducible builds, isolated dependencies, and portable deployment across different devices. For inter-module communication, we use *Redis* as a lightweight, low-latency in-memory data store and message broker to support fast reads/writes and efficient message passing between modules. We use *Protobuf* to define a strict schema for all exchanged messages and to serialize metadata compactly, improving interoperability and reducing transmission overhead. Finally, we store association outputs and trajectory records in *TimescaleDB*, a time-series database built on PostgreSQL, to support persistent storage, efficient time-based queries, and long-term data management.

4. Experiment

4.1. Dataset

4.1.1. RoundaboutHD

RoundaboutHD, introduced in our previous work (Lin et al., 2025b), is a real-world multi-camera vehicle tracking dataset designed for roundabout traffic scenarios. It contains 40 minutes of fully annotated, non-overlapping



Figure 2: Camera layouts and sample views of the datasets used in this work.

multi-camera video captured from four 4K cameras at 15 FPS with geographic layout shown in fig. 2a. The dataset introduces several challenges for MCVT, including: (1) nonlinear vehicle trajectories, (2) frequent occlusions caused by infrastructure and other vehicles, and (3) complex environments like multiple exits and intersections across camera views. And at the time of submission, we are not aware of any published work that has fully addressed these challenges. In this work, we use the RoundaboutHD as the primary dataset to evaluate our proposed EASE-MCVT framework.

4.1.2. CityFlow

CityFlow (Tang et al., 2019) is one of the earliest and most representative open-source MCVT datasets, released by NVIDIA Research in 2019. Its test-set layout is shown in fig. 2b. The dataset contains approximately 3.25 hours of 960p traffic video recorded from 46 cameras across 10 intersections in a medium-sized U.S. city, covering roughly 2.5 km of roadway. In our experiment, we use the training and validation splits (40 cameras) to fine-tune the feature extractor, and the 6-camera test split to evaluate the performance of the proposed EASE-MCVT pipeline.

4.2. Implementation Details

4.2.1. Hardware settings

Due to hardware availability constraints, the EASE-MCVT framework is evaluated in a laboratory environment using a single NVIDIA Jetson AGX Orin 64 GB module as the edge computing device, and a workstation equipped with an Intel 13th Gen i7-13700KF CPU and an NVIDIA GeForce RTX 4070 Ti GPU as the central server, running Ubuntu 20.04.4 LTS. The edge node and the central server are connected via a wired Ethernet link.

In this paper, we emulate a multi-camera deployment by running the edge pipeline per camera, recording the resulting message streams, and replaying multiple streams concurrently to simulate multi-node operation. In parallel, the framework is also being tested for real-world deployment in several European cities.

4.2.2. Hyperparameter settings

Unless otherwise stated, the hyperparameter settings in this section were selected empirically through preliminary experiments, with the aim of balancing accuracy, runtime stability, and practical deployment constraints.

- **Object Detection.** We follow the standard YOLO inference pipeline. The confidence threshold is set to 0.35 and the IoU threshold for non-maximum suppression is 0.40. The image inference size is configured as 1120×672 to improve detection of small, distant vehicles. All remaining settings follow the default YOLO configuration. The motion detector threshold τ_{fg} is set to $1e-4$.
- **Single-Camera Tracking.** We adopt the default parameter settings from BoxMOT (Broström, 2025) for the single-camera tracking module.

- **Geo-Mapping.** The vehicle center-of-gravity height is assumed to be 0.5 meter.
- **Feature Extraction.** For the dynamic workload adjustment scheme, the $Q_{\max}(\Delta)$ is 25 images within 1 second allowed processing window.
- **Single-Camera Rematch.** For refining fragmented single-camera tracklets, we use a temporal threshold of $T_{\text{th}} = 3$ s (45 frames), a spatial threshold of $D_{\text{th}} = 0.2$ of the image width, and a feature-similarity threshold of $F_{\text{th}} = 0.2$.
- **Self-Supervised Camera Link Model.** The bandwidth parameter for the KDE-based transition-time estimation is set to 5.0.
- **Cross-Camera Association.** The association process is executed every $\alpha = 2500$ frames. The cost-function weights are set to $\delta = 1$ and $\epsilon = 1$, and the greedy-matching distance threshold is 0.4.

4.3. EASE-MCVT Framework Performance Evaluation

4.3.1. Edge-Side Performance

4.3.1.1. Object Detection. We evaluate the edge-side vehicle detection module on RoundaboutHD in terms of detection accuracy and runtime. All results are obtained using our fine-tuned YOLOv11-nano detector deployed with TensorRT (FP16) on an NVIDIA Jetson AGX Orin, with an input image size of 1120×672 . Detection accuracy is measured using mean Average Precision (mAP) averaged over IoU thresholds from 0.5 to 0.95. Runtime is reported as the average per-frame inference latency (ms). We compare our detector against the non-fine-tuned YOLOv11X baseline provided with RoundaboutHD; results are summarized in table 1.

Table 1: Edge-side object detection evaluation on RoundaboutHD. $\uparrow$ indicates higher is better, and $\downarrow$ indicates lower is better.

Metric	Cam01		Cam02		Cam03		Cam04	
	Baseline	Ours	Baseline	Ours	Baseline	Ours	Baseline	Ours
mAP@0.5:0.95 $\uparrow$	42.08	56.47	70.39	66.62	74.39	65.44	76.60	51.25
Latency (ms/frame) $\downarrow$	Offline	25.84	Offline	25.68	Offline	25.50	Offline	25.68

Overall, our implementation achieves low inference latency on the edge device while maintaining good detection accuracy for real-time deployment. We do not report motion-detector results on RoundaboutHD because the traffic flow in the videos are continuous and rarely contain idle periods, so frame skipping would provide little benefit.

4.3.1.2. Single Camera Tracking. We evaluate the edge-side single-camera tracking module on RoundaboutHD by comparing our online, edge-deployed tracker against the offline SCT baseline reported with the dataset. We use ByteTrack as the default tracker in EASE-MCVT. Tracking accuracy is measured using IDF1 (Ristani et al., 2016), and runtime is reported as the average per-frame tracking latency (ms) measured on the Jetson AGX Orin. Results for each camera are summarized in table 2.

Table 2: Edge-side single-camera tracking evaluation on RoundaboutHD. $\uparrow$ indicates higher is better, and $\downarrow$ indicates lower is better.

Metric	Cam01		Cam02		Cam03		Cam04	
	Baseline	Ours	Baseline	Ours	Baseline	Ours	Baseline	Ours
IDF1 $\uparrow$	72.60	64.10	90.50	66.30	84.20	55.80	86.00	73.40
Latency (ms/frame) $\downarrow$	Offline	5.48	Offline	5.66	Offline	5.80	Offline	3.23

As expected, the offline baseline achieves higher IDF1, as it can use heavier models and non-real-time processing. In contrast, our edge implementation delivers low per-frame latency (3–6 ms) while maintaining reasonable identity consistency, providing a practical SCT component for real-time deployment.

4.3.1.3. Feature Extraction. We evaluate the feature extraction module on RoundaboutHD in terms of embedding quality and runtime stability. For embedding quality, we adopt the SBS method from the FastReID framework (He et al., 2020), which uses a ResNeXt50 backbone and an input resolution of 256×256 . Following the RoundaboutHD protocol, the model is fine-tuned on the RoundaboutHD image-based ReID training set. The fine-tuned model achieves 99.19 mAP on the RoundaboutHD image-based ReID benchmark, indicating strong discriminative capability in the overhead traffic-surveillance domain.

For runtime, our pipeline performs feature extraction at the tracklet level: the edge node receives per-frame tracklet-update messages and buffers them in a FIFO queue until a tracklet is completed. Because computation is not executed per-frame, per-frame latency is not a meaningful measure of runtime performance. Instead, we assess real-time stability by monitoring the occupancy of the input message queue, which reflects the balance between the arrival rate of per-frame updates and the service rate of the extraction pipeline under the *dynamic workload scheme*. We report the average and maximum queue occupancy (in number of messages) during the RoundaboutHD sequences in table 3.

Table 3: Edge-side feature extraction queue occupancy on RoundaboutHD. The queue capacity is 64 messages.

Metric (messages)	Cam01	Cam02	Cam03	Cam04
Avg. queue occupancy↓	2.25	2.25	2.20	1.39
Max. queue occupancy↓	22	18	16	21

Across all cameras, the queue remains bounded, does not exhibit sustained growth, and stays well below the configured capacity of 64 messages, indicating that the system can keep pace with the incoming message stream under the RoundaboutHD workload. In this work, we treat bounded queue occupancy (and zero overflow events) as evidence that the feature extraction module operates in real time in the sense of stable throughput.

4.3.1.4. Bandwidth Evaluation. In an edge-server architecture, bandwidth is a key scalability constraint: higher transmission rates increase network cost and can amplify queuing delay or message loss under high data volume pressure. To quantify the bandwidth savings of EASE-MCVT, we compare two edge-to-server payload settings: (i) tracklet metadata + trajectory-level embeddings (no images), and (ii) tracklet metadata + cropped JPEGs (image payload included). We measure the total transmitted payload over a 10-minute window and report the corresponding average throughput in MB/s. Results are summarized in Table 4.

Table 4: Edge-to-server bandwidth on RoundaboutHD.

Setting (MB/s)↓	Cam01	Cam02	Cam03	Cam04
Tracklet metadata + embeddings	1.92	1.95	2.45	1.66
Tracklet metadata + cropped JPEGs	10.35	9.12	11.57	9.18

Overall, transmitting embeddings instead of cropped JPEGs reduces the required bandwidth by approximately $5.09\times$ (averaged across cameras). This reduction directly improves scalability by lowering per-camera bandwidth requirement, which makes it easier to support larger camera networks under practical network constraints.

4.3.2. Central Server Performance

4.3.2.1. Self-Supervised Camera Link Model. To evaluate the temporal modeling accuracy of the self-supervised camera link model, we compare the estimated transition-time density with a ground-truth transition-time density computed from the RoundaboutHD annotations. Specifically, for each adjacent camera link, we derive ground-truth transition times from RoundaboutHD ground truth; we then overlay this ground truth density with the kernel density estimate produced by our camera link model. The qualitative comparison is shown in fig. 3.

As shown in fig. 3, the estimated transition-time densities closely match the ground-truth distributions across links, indicating that the proposed camera link model captures the underlying spatial-temporal relationship between adjacent cameras and provides reliable temporal constraints for downstream cross-camera association.

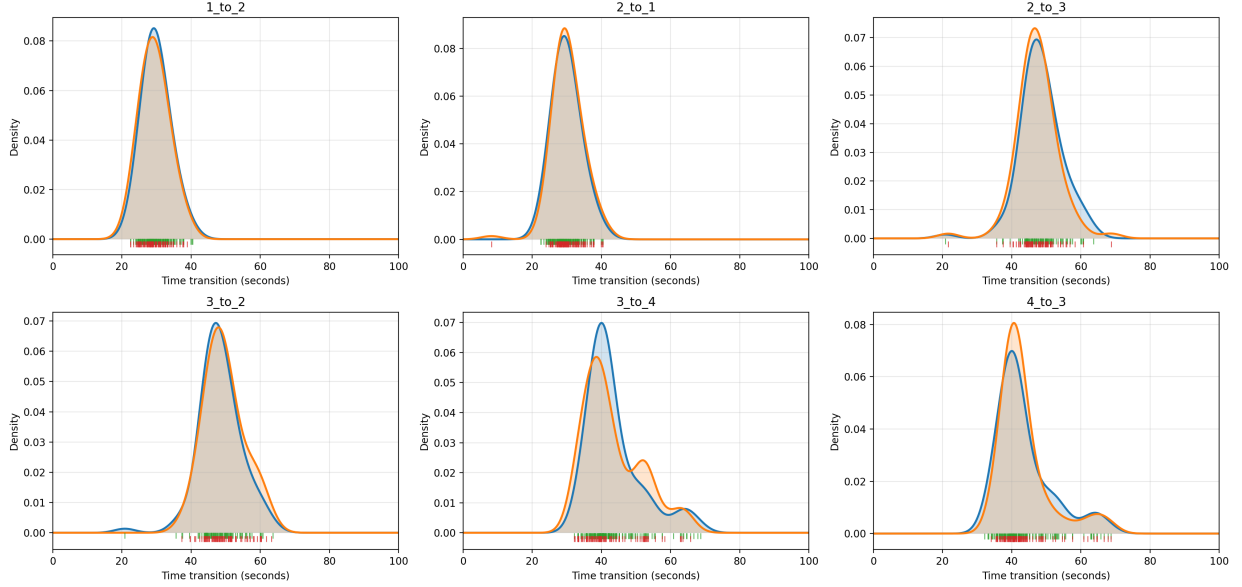


Figure 3: Ground-truth versus estimated transition-time distributions for the self-supervised camera link model. The orange region shows the ground-truth transition-time density, and the blue region shows the estimated KDE from the proposed camera link model. The green vertical bars indicate the estimated transition times, while the red vertical bars indicate the ground-truth transition times.

4.3.2.2. Multi-Camera Tracking. We evaluate the overall multi-camera vehicle tracking performance of EASE-MCVT on both *RoundaboutHD* and *CityFlow*. As shown in table 5, we compare EASE-MCVT with several representative offline state-of-the-art MCVT methods. In addition, to quantify the contribution of our server-side add-on modules, we conduct an ablation study on the *single-camera re-match* module and the *self-supervised camera link model (CLM)*, with results reported in table 6.

Table 5: Comparison of multi-camera vehicle tracking performance on RoundaboutHD and CityFlow.

Dataset	Method	IDF1↑	IDP↑	IDR↑
RoundaboutHD	ELECTRICITY (Qian et al., 2020)	28.14	26.45	30.06
	AIC21-MCVT (Liu et al., 2021)	36.92	36.54	37.30
	AIC22-MCVT (Li et al., 2022)	45.19	37.00	58.04
	EASE-MCVT(Ours)	71.63	80.60	64.44
CityFlow	ELECTRICITY (Qian et al., 2020)	45.85	-	-
	AIC21-MCVT (Liu et al., 2021)	80.95	85.69	76.70
	AIC22-MCVT (Li et al., 2022)	84.37	89.00	80.20
	EASE-MCVT(Ours)	55.17	75.99	43.31

For the overall MCVT comparison in table 5, EASE-MCVT achieves the best performance on *RoundaboutHD*, reaching an IDF1 score of 71.63%. This result is consistent with our design goal, indicating that the proposed combination of dataset-adapted edge-side modules and server-side association refinements can achieve strong tracking accuracy on this real-world roundabout dataset without sacrificing deployment practicality. On *CityFlow*, EASE-MCVT achieves 55.17% IDF1, which is higher than *ELECTRICITY* but lower than *AIC21-MCVT* and *AIC22-MCVT*. We note that these comparison methods were developed as offline systems specifically for the AI City Challenge setting on *CityFlow*, whereas EASE-MCVT is designed primarily for real-time and scalable deployment rather than maximizing offline benchmark performance. From this perspective, although EASE-MCVT does not match the best offline results on *CityFlow*, its performance still indicates reasonable cross-dataset transferability without dataset-specific

Table 6: Ablation study of add-on modules on RoundaboutHD and CityFlow.

Dataset	Baseline	+SCT Re-Match	+CLM	IDF1↑	IDP↑	IDR↑
RoundaboutHD	✓	-	-	64.92	74.79	57.34
	✓	✓	-	68.75 (+3.83)	77.42 (+2.63)	61.84 (+4.50)
	✓	-	✓	66.64 (+1.72)	77.75 (+2.96)	58.30 (+0.96)
	✓	✓	✓	71.63 (+6.71)	80.60 (+5.81)	64.44 (+7.10)
CityFlow	✓	-	-	43.66	40.72	47.06
	✓	✓	-	43.66 (+0.00)	41.03 (+0.31)	46.47 (-0.59)
	✓	-	✓	54.51 (+10.85)	75.40 (+34.68)	42.68 (-4.38)
	✓	✓	✓	55.17 (+11.51)	75.99 (+35.27)	43.31 (-3.75)

adaptation, while preserving the practical advantages of a deployment-oriented framework.

Beyond tracking accuracy, our framework also demonstrates stable runtime behavior during online operation. Following the same queue-based evaluation methodology used for the feature-extraction module, we monitor the server-side input queue occupancy during multi-camera tracking. On *RoundaboutHD*, the maximum queue occupancy is 12 and the average queue occupancy is 0.21. On *CityFlow*, the maximum queue occupancy increases to 53 and the average queue occupancy to 1.39, reflecting the heavier workload introduced by the larger camera network and denser traffic. Importantly, we observe zero queue overflow events in both settings. This indicates that, under the tested conditions, the central server can process incoming tracklet updates at least as fast as they arrive, providing evidence that EASE-MCVT supports real-time throughput in the sense of stable end-to-end operation. In contrast, the compared baseline methods are designed for offline processing and therefore do not target this deployment setting.

The ablation results in table 6 further verify the effectiveness of the proposed server-side modules. On *RoundaboutHD*, both add-on modules improve performance over the baseline, and the combination of *SCT Re-Match* and *CLM* yields the best result, improving IDF1 from 64.92% to 71.63%. This indicates that reconnecting fragmented single-camera tracklets and introducing learned spatial-temporal constraints are both beneficial for downstream cross-camera association. On *CityFlow*, the overall trend is similar: adding *CLM* produces a substantial gain, increasing IDF1 from 43.66% to 54.51%, and combining both modules further improves IDF1 to 55.17%. In contrast, the *SCT Re-Match* module alone brings only marginal improvement on this dataset. A likely reason is that the edge-side SCT outputs on *CityFlow* are less reliable due to the more challenging visual conditions, including lower video quality, denser traffic, and more frequent occlusions, which limit the amount of recoverable fragmentation through conservative same-camera re-linking. Overall, the ablation study demonstrates that both add-on modules contribute positively to the proposed framework and validates the effectiveness of their design in improving multi-camera tracking performance.

5. Conclusion

This work presents EASE-MCVT, an edge-server architecture based multi-camera vehicle tracking framework for real-world ITS deployment. To enable real-time operation and city-scale scalability, EASE-MCVT combines algorithm-level and system-level adaptations across both the edge and server sides. On the edge side, it integrates a fine-tuned YOLO detector, a lightweight motion-triggering strategy, geo-mapping for additional spatial evidence, and tracklet-level feature extraction with a dynamic workload scheme to stabilize latency under varying traffic density. On the server side, it incorporates a single-camera re-match module to reduce track fragmentation, a self-supervised camera link model to learn spatial-temporal constraints for more reliable cross-camera association and an online triggering scheme to handle asynchronous live streams. At the system level, the framework is implemented within our industrial partner’s SAE-ENGINE to standardize deployment, communication, and data management. Experimental results on public benchmarks show that EASE-MCVT achieves real-time throughput on resource-constrained edge hardware while maintaining strong tracking accuracy, reaching 71.63% IDF1 on *RoundaboutHD*, which is the best result among the compared methods, and 55.17% on *CityFlow*. These results demonstrate that EASE-MCVT is a

practical and scalable solution for real-world multi-camera vehicle tracking, and an important step toward robust city-scale ITS deployment.

Acknowledgment

Yuqiang Lin and Sam Lockyer are supported by a scholarship from the EPSRC Centre for Doctoral Training in Advanced Automotive Propulsion Systems (AAPS) under project EP/S023364/1.

References

- Aharon, N., Orfaig, R., Bobrovsky, B.Z., 2022. Bot-sort: Robust associations multi-pedestrian tracking. arXiv preprint arXiv:2206.14651 .
- Amosa, T.I., Sebastian, P., Izhar, L.I., Ibrahim, O., Ayinla, L.S., Bahashwan, A.A., Bala, A., Samaila, Y.A., 2023. Multi-camera multi-object tracking: A review of current trends and future advances. *Neurocomputing* 552, 126558.
- Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., et al., 2023. Qwen technical report. arXiv preprint arXiv:2309.16609 .
- Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B., 2016. Simple online and realtime tracking, in: 2016 IEEE international conference on image processing (ICIP), Ieee. pp. 3464–3468.
- Broström, M., 2025. BoxMOT: Pluggable sota multi-object tracking modules for segmentation, object detection, and pose estimation. <https://github.com/mikel-brostrom/boxmot>. Commit version used: v13.0.17 (July 2025).
- Gaikwad, B., Karmakar, A., 2021. Smart surveillance system for real-time multi-person multi-camera tracking at the edge. *Journal of real-time image processing* 18, 1993–2007.
- Gerum, R., Richter, S., Winterl, A., Fabry, B., Zitterbart, D., 2017. Cameratransform: a scientific python package for perspective camera corrections. arXiv preprint arXiv:1712.07438 .
- Girshick, R., 2015. Fast r-cnn, in: Proceedings of the IEEE international conference on computer vision, pp. 1440–1448.
- Gong, T., Zhu, L., Yu, F.R., Tang, T., 2023. Edge intelligence in intelligent transportation systems: A survey. *IEEE Transactions on Intelligent Transportation Systems* 24, 8919–8944.
- He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition, in: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770–778.
- He, L., Liao, X., Liu, W., Liu, X., Cheng, P., Mei, T., 2020. Fastreid: A pytorch toolbox for general instance re-identification. arXiv preprint arXiv:2006.02631 .
- Holla, A., MM, M.P., Verma, U., Pai, R.M., 2025. Vehicle re-identification and tracking: Algorithmic approach, challenges and future directions. *IEEE Open Journal of Intelligent Transportation Systems* .
- Hsu, H.M., Huang, T.W., Wang, G., Cai, J., Lei, Z., Hwang, J.N., 2019. Multi-camera tracking of vehicles based on deep features re-id and trajectory-based camera link models., in: CVPR workshops, pp. 416–424.
- Ke, R., Liu, C., Yang, H., Sun, W., Wang, Y., 2022. Real-time traffic and road surveillance with parallel edge intelligence. *IEEE Journal of Radio Frequency Identification* 6, 693–696.
- Ke, R., Zhuang, Y., Pu, Z., Wang, Y., 2020. A smart, efficient, and reliable parking surveillance system with edge artificial intelligence on iot devices. *IEEE Transactions on Intelligent Transportation Systems* 22, 4962–4974.
- Khanam, R., Hussain, M., 2024. Yolov11: An overview of the key architectural enhancements. arXiv preprint arXiv:2410.17725 .

- Li, F., Wang, Z., Nie, D., Zhang, S., Jiang, X., Zhao, X., Hu, P., 2022. Multi-camera vehicle tracking system for ai city challenge 2022, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 3265–3273.
- Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L., 2014. Microsoft coco: Common objects in context, in: *Computer vision—ECCV 2014: 13th European conference, zurich, Switzerland, September 6–12, 2014, proceedings, part v 13*, Springer. pp. 740–755.
- Lin, Y., Lockyer, S., Evans, A., Zarbock, M., Zhang, N., 2025a. Ablation study for multicamera vehicle tracking using cityflow dataset, in: *Seventeenth International Conference on Machine Vision (ICMV 2024)*, SPIE. pp. 246–253.
- Lin, Y., Lockyer, S., Sui, M., Gan, L., Stanek, F., Zarbock, M., Li, W., Evans, A., Zhang, N., 2025b. Roundaboutd: High-resolution real-world urban environment benchmark for multi-camera vehicle tracking. *arXiv preprint arXiv:2507.08729*.
- Lin, Y., Lockyer, S., Zhang, N., 2025c. City-scale multi-camera vehicle tracking system with improved self-supervised camera link model. URL: <https://arxiv.org/abs/2405.11345>, *arXiv:2405.11345*.
- Liu, C., Yang, H., Zhu, M., Wang, F., Vaa, T., Wang, Y., 2023. Real-time multi-task environmental perception system for traffic safety empowered by edge artificial intelligence. *IEEE Transactions on Intelligent Transportation Systems* 25, 517–531.
- Liu, C., Zhang, Y., Luo, H., Tang, J., Chen, W., Xu, X., Wang, F., Li, H., Shen, Y.D., 2021. City-scale multi-camera vehicle tracking guided by crossroad zones, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4129–4137.
- Liu, X., Liu, W., Mei, T., Ma, H., 2017. Provid: Progressive and multimodal vehicle reidentification for large-scale urban surveillance. *IEEE Transactions on Multimedia* 20, 645–658.
- NVIDIA, 2026. Nvidia tensorrt documentation. <https://docs.nvidia.com/deeplearning/tensorrt/latest/>.
- Qian, Y., Yu, L., Liu, W., Hauptmann, A.G., 2020. Electricity: An efficient multi-camera vehicle tracking system for intelligent city, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pp. 588–589.
- Ristani, E., Solera, F., Zou, R., Cucchiara, R., Tomasi, C., 2016. Performance measures and a data set for multi-target, multi-camera tracking, in: *European conference on computer vision*, Springer. pp. 17–35.
- Shen, F., Xie, Y., Zhu, J., Zhu, X., Zeng, H., 2023. Git: Graph interactive transformer for vehicle re-identification. *IEEE Transactions on Image Processing* 32, 1039–1051.
- Tang, Z., Naphade, M., Liu, M.Y., Yang, X., Birchfield, S., Wang, S., Kumar, R., Anastasiu, D., Hwang, J.N., 2019. Cityflow: A city-scale benchmark for multi-target multi-camera vehicle tracking and re-identification, in: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 8797–8806.
- Tian, Y., Ye, Q., Doermann, D., 2025. Yolov12: Attention-centric real-time object detectors. *arXiv preprint arXiv:2502.12524*.
- Varghese, R., Sambath, M., 2024. Yolov8: A novel object detection algorithm with enhanced performance and robustness, in: *2024 International conference on advances in data engineering and intelligent computing systems (ADICS)*, IEEE. pp. 1–6.
- Wang, Y., Zhang, D., Liu, Y., Dai, B., Lee, L.H., 2019. Enhancing transportation systems via deep learning: A survey. *Transportation research part C: emerging technologies* 99, 144–163.
- Wojke, N., Bewley, A., Paulus, D., 2017. Simple online and realtime tracking with a deep association metric, in: *2017 IEEE international conference on image processing (ICIP)*, IEEE. pp. 3645–3649.

- Wu, M., Qian, Y., Wang, C., Yang, M., 2021. A multi-camera vehicle tracking system based on city-scale vehicle re-id and spatial-temporal information, in: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 4077–4086.
- Xie, S., Girshick, R., Dollár, P., Tu, Z., He, K., 2017. Aggregated residual transformations for deep neural networks, in: *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1492–1500.
- Yang, H.F., Cai, J., Liu, C., Ke, R., Wang, Y., 2023. Cooperative multi-camera vehicle tracking and traffic surveillance with edge artificial intelligence and representation learning. *Transportation research part C: emerging technologies* 148, 103982.
- Yang, X., Ye, J., Lu, J., Gong, C., Jiang, M., Lin, X., Zhang, W., Tan, X., Li, Y., Ye, X., et al., 2022. Box-grained reranking matching for multi-camera multi-target tracking, in: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 3096–3106.
- Zang, Y., Li, W., Han, J., Zhou, K., Loy, C.C., 2025. Contextual object detection with multimodal large language models. *International Journal of Computer Vision* 133, 825–843.
- Zhang, X., Izquierdo, E., 2019. Real-time multi-target multi-camera tracking with spatial-temporal information. 2019 *IEEE Visual Communications and Image Processing (VCIP)* , 1–4.
- Zhang, X., Yu, H., Qin, Y., Zhou, X., Chan, S., 2024. Video-based multi-camera vehicle tracking via appearance-parsing spatio-temporal trajectory matching network. *IEEE Transactions on Circuits and Systems for Video Technology* 34, 10077–10091.
- Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., Wang, X., 2022. Bytetrack: Multi-object tracking by associating every detection box, in: *European conference on computer vision*, Springer. pp. 1–21.
- Zheng, Z., Jiang, M., Wang, Z., Wang, J., Bai, Z., Zhang, X., Yu, X., Tan, X., Yang, Y., Wen, S., et al., 2020. Going beyond real data: A robust visual representation for vehicle re-identification, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pp. 598–599.
- Zhou, X., Ke, R., Yang, H., Liu, C., 2021. When intelligent transportation systems sensing meets edge computing: Vision and challenges. *Applied Sciences* 11, 9680.
- Zhuang, H., Chen, X., Lan, J., Wang, Y., Lu, Z., Wang, C., Yang, M., 2024. A real-time framework of multi-camera vehicle tracking system, in: *2024 IEEE International Conference on Real-time Computing and Robotics (RCAR)*, IEEE. pp. 149–154.
- Zivkovic, Z., 2004. Improved adaptive gaussian mixture model for background subtraction, in: *Proceedings of the 17th International Conference on Pattern Recognition, 2004. ICPR 2004.*, IEEE. pp. 28–31.